TECHNICAL DOCUMENTATION

HORIZON GRID

Linux Technical Documentation

Version: 0.2.5

Documentation prepared: 2026-08-23

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID Linux Technical Documentation

This document describes the Linux packaging layer for HORIZON GRID: how the `.deb` package, systemd unit, and CLI setup wizard are built, installed, and maintained. It does not cover the application's own architecture (backend/frontend/Docker Compose services) beyond what's needed to understand what the packaging layer wraps — see the existing application and security documentation for that.

Packaging Architecture Decision

HORIZON GRID on Linux is distributed as a `.deb` package (`horizon-grid_0.2.2_amd64.deb`) that installs a systemd unit and a `horizon-grid` CLI, including a terminal setup wizard. This is the direct Linux analog of the Windows installer + WinForms wizard + Docker Compose combination that already exists for that platform.

Why not an AppImage. This was a deliberate decision, confirmed with the user before packaging work began. HORIZON GRID is a multi-container Docker Compose platform — Postgres, Redis, Neo4j, OpenSearch, a FastAPI backend, a Next.js frontend, and two Celery workers — accessed via a web browser, not a single GUI executable. An AppImage exists to wrap one GUI binary; that packaging model does not fit an application whose actual runtime is a set of orchestrated containers. Wrapping only, say, a launcher binary in an AppImage while the real workload runs in Docker Compose would misrepresent what is actually being packaged.

Why `.deb` + systemd + CLI wizard. This combination is the closest real analog of what already ships on Windows:

- The Windows installer doesn't reimplement HORIZON GRID as a native Windows service — Docker Compose is the actual runtime there too. The `.deb` plays the same role: it stages files and registers a systemd unit, but the systemd unit's job is simply to invoke Docker Compose, not to run the application itself.
- The WinForms setup wizard is replaced by a terminal program (`linux/wizard/setup_wizard.py`) that performs the identical steps and the identical HTTP calls against the backend, just without a GUI toolkit — appropriate for a server/sysadmin-oriented Linux install.
- `apt remove` / `apt purge` map onto Linux's own idiomatic uninstall model, and turn out to be a cleaner, more explicit two-tier analog of the Windows uninstaller's "keep data" vs "delete everything" paths.

File Layout

The installed layout follows FHS conventions and maps directly onto the Windows Program Files / ProgramData split.

Linux path	Purpose	Windows analog
`/opt/horizon-grid/app/`	The application itself: `backend/`, `frontend/`, `docker-compose.yml`, `docker-compose.prod.yml`, `docs/` — read-mostly	Program Files (application binaries/assets)
`/etc/horizon-grid/.env`	Real configuration/secrets, `root:root`, `chmod 600` (verified live)	ProgramData config, locked to Administrators+SYSTEM
`/var/lib/horizon-grid/backups/`	`pg_dump` snapshots	ProgramData backups folder
`/var/log/horizon-grid/setup.log`	Setup/wizard log	ProgramData `logs\setup.log`
`/usr/bin/horizon-grid`	Single CLI entrypoint	Start Menu shortcuts / installer-provided scripts
`/lib/systemd/system/horizon-grid.service`	systemd unit (registered via daemon-reload, never auto-started)	Windows service registration (also not auto-started before configuration)
`/usr/share/applications/horizon-grid.desktop`	Desktop menu launcher, `Exec=horizon-grid open`; no custom icon yet, falls back to a generic one	Start Menu shortcut

One deliberate parity detail: both platforms' Compose project directory has the same basename (`app`), so both produce the identical Docker Compose project label `com.docker.compose.project=app`. This is by design, not coincidence, and it's what makes the label-based remove/purge logic (below) behave identically on both platforms.

Building the Package

The `.deb` is built by a Linux-specific build script that stages the application tree and application metadata (control file, postinst/postrm scripts, systemd unit, desktop file) into a package layout and invokes standard Debian packaging tooling (`dpkg-deb`) to produce the final artifact. This must run on a real Debian/Ubuntu host (or container) — `dpkg-deb` is a Debian-family tool with no Windows equivalent, the same reason the Windows installer can only be built with Inno Setup's `ISCC.exe` on Windows.

Why it doesn't vendor dependencies. The package does not bundle `node_modules`, Python pip dependencies, or pre-built Docker images. All of those install the same way on every platform: inside the containers, the first time `docker compose up --build` runs. This is why the package is small (~6.1 MB / 5.8 MiB) despite the application being substantial — the same reason the ~69 MB Windows installer doesn't contain Docker images either. Both installers ship orchestration and configuration, not the workload's runtime dependencies.

A real bug found and fixed in this build process: the build script's `rsync` of `backend/` initially swept up two host-only Python virtualenvs (`.venv`, `.venv_test` — about 21,700 files combined) that have no place in a shipped package, since dependencies install inside the container at image-build time, exactly as on Windows. This was fixed by excluding both directories in the Linux build script. Separately — and not fixed in this pass, since it lives in a Windows-side file outside the scope of this Linux effort — the Windows installer's `installer.iss` `Excludes` list has the same latent gap and could pick up the same two folders if they happen to exist on the machine building that installer. This is flagged here as a discovered, non-blocking hardening item for the Windows installer specifically.

The systemd Unit

`/lib/systemd/system/horizon-grid.service` is registered (via `daemon-reload`) at package install time but is **never** auto-enabled or auto-started by the package itself. Nothing starts until the administrator runs the setup wizard — this matches the Windows installer's own behavior, where nothing auto-starts before configuration either.

The unit wraps Docker Compose rather than reimplementing the application as a native systemd service, consistent with the packaging decision above: Docker Compose is the real runtime on both platforms, and the unit's only job is to bring that Compose project up and keep systemd's view of "is HORIZON GRID up" accurate.

It is configured as `Type=oneshot` with `RemainAfterExit=yes`. This is the correct shape for a unit whose start action is "run a command that hands off to a long-lived external process group it doesn't itself hold open" — `docker compose up -d` returns once the containers are launched in detached mode; the unit process itself does not stay in the foreground supervising them (the containers are supervised by the Docker daemon, not by systemd). `Type=oneshot` tells systemd the start command is expected to exit; `RemainAfterExit=yes` tells systemd to still consider the unit "active" after that exit, rather than treating the exit as a failure or an immediate return to "inactive". This lets `systemctl status horizon-grid`, `enable`, and `stop` behave sensibly for a unit that is really just a thin wrapper delegating persistent state to Docker itself. `ExecStart` / `ExecStop` / `ExecReload` each call the `horizon-grid` CLI (`start` / `stop` / `restart`) rather than a raw `docker compose` command directly, so the unit always goes through the same `.env`-sync and health-wait logic every other entry point uses.

The CLI and Setup Wizard

CLI commands

`horizon-grid <command>` (most require `sudo`): `configure` (runs the setup wizard), `start`, `stop`, `restart`, `status`, `open` (start-if-needed and open the browser), `backup`, `diagnostics`, `check` (prerequisites), `uninstall` (prints `apt remove` / `purge` guidance), `version`.

Wizard flow

`linux/wizard/setup_wizard.py` is a terminal program that replaces the Windows WinForms wizard (`windows/wizard/Setup-Wizard.ps1`) but follows the **same real flow and makes the same real HTTP calls**, in this order:

Welcome → Administrator Account → AI Configuration → Threat Intelligence Providers → Network Ports → Summary → write `/etc/horizon-grid/.env` (`0600`, `root:root`) → `docker compose up -d --build` → wait for `GET /health/detailed` (up to 3 minutes -- confirms Postgres is genuinely reachable before the next step writes to it, not just that the process has started) → register the admin account via `POST /api/v1/auth/register` (bootstrap-only: succeeds only while the users table is empty, so this only actually creates an account on a genuinely fresh install) → confirm via `POST /api/v1/auth/login`.

On a **reconfigure** of an existing install, re-entering the existing admin email and password signs in for that session and enables live "Test Connection" calls (`POST /api/v1/providers/{id}/test`, `POST /api/v1/ai/test`) — exactly as on Windows. During a **fresh** install's provider/AI pages, Test Connection cannot work yet because no backend is running at that point in the flow. This is an application-level fact true on both platforms, not a Linux-specific limitation.

Providers configured by the wizard

The wizard writes directly to `.env`, matching `windows/scripts/Write-EnvFile.ps1`'s exact key list. This covers two categories of AI/provider configuration:

- **AI backends chosen and configured directly in the wizard's own AI Configuration page**: all eleven backends (Ollama, Anthropic, Amazon Bedrock, Google Gemini, Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, OpenRouter) are wizard-configurable choices (`linux/wizard/setup_wizard.py`'s own `backends` list), written straight to `.env` (`AI_BACKEND` plus that backend's own key/model fields).
- **Threat-intelligence providers with a dedicated wizard field**: VirusTotal, AbuseIPDB, AlienVault OTX, NIST NVD (optional key), the shared abuse.ch key (activates URLhaus + ThreatFox + MalwareBazaar together), Hybrid Analysis, Censys (Personal Access Token + Organization ID, both required together), and PhishTank (optional key).

Providers needing no credential at all (crt.sh, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, the Internet Intelligence Collector) and the newer, runtime-only threat-intelligence providers that have no wizard field of their own (urlscan.io, Google Safe Browsing) are instead configured and managed after install from the app's own Providers page (sign in → Providers). This is identical on both platforms, since it's a database-backed runtime configuration feature of the application itself, not of either installer — Groq is a wizard-configurable **AI backend** choice, distinct from that runtime-only provider category.

Ports

Identical to Windows, because it's the same `docker-compose.yml`:

- **Published on all interfaces** (LAN-reachable by design): Frontend `3000`, Backend `8000`.
- **Loopback-only (`127.0.0.1`)**, confirmed live via `docker compose ps` output during testing (bindings such as `127.0.0.1:27475->7474/tcp`): Postgres `5433`, Redis `6379`, Neo4j HTTP `7475` / Bolt `7688`, OpenSearch `9200`.

The wizard detects port conflicts and offers the next free port, same as on Windows.

Remove vs Purge

Linux offers a two-tier uninstall that is a more idiomatic analog of the Windows uninstaller's two paths:

- **`sudo apt remove horizon-grid`** — keeps `/etc/horizon-grid`, `/var/lib/horizon-grid`, and all Docker volumes (Postgres/Neo4j/OpenSearch data). Reinstalling later picks up right where you left off. Confirmed live in both the three-distro test suite and a separate isolated default-project-name test: containers stop, volumes and config survive.
- **`sudo apt purge horizon-grid`** — deletes everything: stops and removes every container and volume labeled for the Compose project, then deletes `/etc/horizon-grid`, `/var/lib/horizon-grid`, and `/var/log/horizon-grid`. No undo. Confirmed live via an isolated Docker-in-Docker test using the real default project name: 12/12 checks passed, including "all containers removed" and "all volumes removed".

Cleanup is label-based (`com.docker.compose.project=<project>`), not a hardcoded container-name guess. This is what makes purge/remove correct even if an administrator customizes `COMPOSE_PROJECT_NAME` — the cleanup logic finds and acts on whatever is actually labeled for the project in effect, rather than assuming a fixed name.

A real bug in this area was found and fixed: `apt purge` / `apt remove` initially left one small file behind, `/opt/horizon-grid/app/.env` — a runtime-written copy that dpkg's manifest never tracked — which blocked full removal of that directory ("directory not empty so not removed"). This was fixed in the package's `postrm` script. It is the direct Linux analog of a documented Windows uninstaller fix for the exact same underlying problem (an untracked, runtime-written `.env` copy blocking cleanup), just via a different mechanism: dpkg simply not tracking the file, versus Windows's ACL blocking Inno Setup's own cleanup.

The Docker Compose v2 Plugin Gotcha

A real, live-confirmed finding on all three tested distributions (Debian 12, Ubuntu 22.04, Ubuntu 24.04): the distribution's own `docker.io` package does **not** include the Docker Compose v2 plugin this application requires. Installing `docker.io` and then running `docker compose version` fails with `docker: 'compose' is not a docker command`. Debian 12's own repositories don't even have a `docker-compose-v2` package to fall back to — only the deprecated standalone v1 `docker-compose` binary.

The correct, working install method — what the package's `control` file `Recommends` and what the Installation Guide must tell users to run — is Docker's own official convenience script:

```
curl -fsSL https://get.docker.com | sh
```

(or manually adding Docker's own apt repository per https://docs.docker.com/engine/install/). Do **not** tell users to simply `apt install docker.io` and stop there; this will silently leave them without a working `docker compose` command.

Backup Container-Resolution Bug

`horizon-grid backup` initially used a hardcoded container name (`app-postgres-1`), mirroring Windows's own `Backup-Database.ps1` exactly. This silently skipped the backup — logging a misleadingly reassuring "Postgres container is not running — skipping backup" message as if it were a successful no-op — whenever a non-default Compose project name was in use.

This was fixed to resolve the real container via `docker compose ps -q postgres` (label/service-based, respecting whatever project name is actually in effect). Confirmed live afterward: a real 34,786-byte `pg_dump` SQL file was produced. This is a small, Linux-specific robustness improvement over the Windows script's hardcoded name; it does not change or regress the Windows side, whose hardcoded name remains correct for its own always-default-project-name usage.

Security Properties

- `/etc/horizon-grid/.env` is `0600`, owned `root:root` — verified live on all three tested distros.
- Every service/wizard script requires root (checks `id -u` / `os.geteuid()`, refuses otherwise) — the direct analog of Windows's UAC-elevation requirement, without needing an equivalent to Windows's filtered-token nuance, since Linux root/sudo has no comparable split-token gap.
- `horizon-grid diagnostics` redacts every credential in its bundled `.env` copy to `KEY=***REDACTED***(set, N chars)` or `KEY=(empty)` — the real value never appears, matching Windows's `Diagnostics.ps1` exactly.
- The four datastores (Postgres, Redis, Neo4j, OpenSearch) bind to `127.0.0.1` only. This comes from `docker-compose.yml` itself — the same file used on both platforms — and is unchanged by anything Linux-specific.
- Frontend and backend are intentionally published on all interfaces for LAN reachability, same as on Windows.
- No firewall is guaranteed present on Linux the way Windows Firewall always is. The wizard/CLI does a best-effort detection and configuration of `ufw` or `firewalld` if active, and clearly reports what it did (or that nothing was detected) rather than assuming a firewall exists.
- No hardcoded credentials appear anywhere in any script. Test Connection semantics are identical to Windows (see the wizard section above).

Test Methodology

Each of the three supported distributions — **Ubuntu 24.04 LTS, Ubuntu 22.04 LTS, and Debian 12 (bookworm)** — was genuinely tested this session, disclosed honestly here rather than implying a bare-metal test lab:

Tests ran inside real Ubuntu/Debian Docker containers on the build host (Docker Desktop's own Linux VM on Windows), using a **sibling-container pattern**: the host's real Docker socket was mounted into the test container, so Docker Compose commands issued from inside it drove the real outer Docker daemon and created real, fully genuine Linux containers. This is real Linux, real `apt install`, real service behavior — not mocked — but it is not a bare-metal machine, and no literal "double-click a desktop icon on a live GNOME/KDE session" verification was possible, since no real desktop GUI environment was available.

Where a test needed the real, default Compose project name (rather than a collision-avoiding override), it ran instead inside a throwaway, fully isolated **Docker-in-Docker** daemon, specifically so it couldn't collide with any other real instance running on the shared build host.

Other distributions may well work — this is a standard Docker Compose + systemd application with nothing distro-specific in the actual runtime — but they have not been tested and must not be described as supported.

Test results

Each distribution ran the identical 35-step, real end-to-end test: real `apt install`, a real setup wizard run building real Docker images, real admin registration/login, a real IOC investigation of `8.8.8.8` through the full pipeline via SSE (`POST /api/v1/lookup/stream`) to completion, real `GET /api/v1/providers/health` and `GET /api/v1/dashboard/kpis` calls, a real backup, stop/start with data-persistence verification, `apt remove` with volume-preservation verification, reinstall with data-recovery verification, and `apt purge`.

Distribution	Pass	Fail
Debian 12 (bookworm)	33	2
Ubuntu 24.04 LTS	33	2
Ubuntu 22.04 LTS	33	2

The same 2 failures appeared on all three, both in the final `apt purge` step ("left N volume(s)/container(s) behind"). Both are a known, explained artifact of the test harness itself, not a real product defect: those three runs deliberately used a non-default `COMPOSE_PROJECT_NAME` to avoid colliding with another real instance running on the same shared build host at the same time — but the purge script's label-based cleanup, correctly, only ever targets the project's real label, so a non-default project name used purely for test isolation is not what gets cleaned up under that scenario.

A separate, dedicated test using the actual default project name (`app`), inside a fully isolated Docker-in-Docker daemon with zero collision risk, confirmed the purge/remove logic is fully correct under realistic conditions: **12 PASS, 0 FAIL**, including "all containers removed" and "all volumes removed".

AI-safety finding (positive proof point, not a bug)

A real, live `8.8.8.8` investigation (screenshot-capture instance, Debian 12) produced a deterministic Threat Score of 26/100 ("Low" severity) from the scoring engine. The configured local AI backend (Ollama, model `gemma2:2b`, a small 2B-parameter model) proposed

an internally inconsistent "malicious" verdict for that same 26.0 score. The platform's documented consistency validator correctly rejected that verdict twice (one retry), and the platform fell back to the honest "Unknown" verdict rather than accept the AI's wrong claim or invent something else — visibly, on screen: "AI-generated assessment unavailable (generation error). See individual provider results and summaries below for raw findings." This is the same safety mechanism already documented for this project's EICAR-hallucination incident, now independently confirmed on Linux with a different, smaller/weaker AI model.

Provider behavior confirmed on Linux

In the same `8.8.8.8` investigation, providers needing no credential returned real, live results: Spamhaus DBL/ZEN (status `ok`, real DNS-based blocklist query), WHOIS/RDAP (status `ok`, real RDAP lookup), and the Internet Intelligence Collector (status `ok`, real live OSINT crawl, including real GitHub repositories referencing the ASN). Providers needing a credential correctly reported `not_configured` (VirusTotal, AlienVault OTX, Censys, URLhaus, ThreatFox, AbuseIPDB) — none were configured in this throwaway test install, deliberately, to avoid using real production API keys inside an ephemeral test container.

One of the Internet Intelligence Collector's four OSINT sub-sources (its paste-dump search module) hit a transient DNS/network failure reaching one paste-site host during this run — correctly isolated per-source (confirmed via backend logs, with the traceback originating in `app/crawler/sources/pastebin_search.py` specifically) with zero effect on the other three sub-sources (GitHub, Reddit, and RSS feeds all succeeded, with logs showing real 200/301/302 responses from `blog.talosintelligence.com`, `crowdstrike.com`, and `unit42.paloaltonetworks.com`) or on the Collector's own overall `ok` status. This is an ordinary transient network hiccup hitting one of four redundant sub-sources, handled exactly per the provider's documented per-source isolation design — not a Linux-specific defect.

Cross-Platform Parity and Windows Regression

At the application layer, parity between Linux and Windows is effectively 100%, because it is the literal same Docker images and code running on both platforms — IOC investigation, threat scoring, AI backend switching, provider switching, the admin UI, Provider Health, the Executive Dashboard, cases, the IOC Basket, exports (JSON/Markdown work; PDF/CSV honestly report "Export format not yet available" on **both** platforms, not a Linux gap), and the Security Assessment tools are all identical, as confirmed live on Linux above.

The only real differences are in the outer packaging/installer layer:

- a terminal wizard instead of a WinForms GUI wizard;
- a single `horizon-grid` command instead of a Start Menu shortcut group;
- best-effort `ufw` / `firewalld` detection instead of a guaranteed Windows Firewall rule;
- the `0600` `root:root` permission lock as the direct (and only) analog of Windows's ACL-lock-plus-optional-DPAPI-encryption. DPAPI has no meaningful Linux equivalent and was never load-bearing on Windows either — the ACL/permission lock was always the real protection there too, per the existing Windows-side security documentation.

Windows regression: zero files under `backend/`, `frontend/`, `docker-compose.yml`, `docker-compose.prod.yml`, or `windows/` were created, modified, or deleted during this Linux packaging effort, verified directly via file timestamps and content. Every new file lives under the new, additive `linux/` directory, which nothing in the Windows installer or the application runtime references. This is reported as a structural/file-level non-regression guarantee. A live functional click-through re-test of the currently installed Windows platform itself was **not** performed in this pass, honestly, because it requires interactive Administrator elevation (UAC) that could not be granted unattended.

Reproducing the Build

To reproduce the `.deb` build on another machine:

1. Use a real Debian- or Ubuntu-family host (not cross-built elsewhere), since the build produces a conventional `.deb` via `dpkg-deb` and the package's control/maintainer scripts — the same reason the Windows installer can only be built with Inno Setup on Windows.
2. Run the Linux build script from the `linux/` directory of the source tree. It stages `backend/`, `frontend/`, `docker-compose.yml`, `docker-compose.prod.yml`, and `docs/` into the package layout under `/opt/horizon-grid/app/`, along with the systemd unit, desktop file, and `postinst` / `postrm` scripts.
3. Confirm the build excludes host-only Python virtualenvs (`.venv`, `.venv_test`) from `backend/` — this exclusion is already present in the Linux build script as a fix for the bug described above; if reproducing or modifying the script, verify this exclusion is still in place before packaging.
4. Do not attempt to vendor `node_modules`, pip dependencies, or pre-built Docker images into the package — these are intentionally left to install inside the containers on first `docker compose up --build`, matching the Windows installer's own approach and keeping the package small.
5. After building, verify the resulting `.deb`'s size is in the same ~6 MB range as `horizon-grid_0.2.2_amd64.deb` (5.8 MiB); a much larger artifact likely indicates a vendored-dependency or virtualenv regression.
6. Before testing, ensure the target test environment has Docker Compose v2 available — installing only `docker.io` from distro repos will not provide `docker compose`; use Docker's official convenience script (`curl -fsSL https://get.docker.com | sh`) as described above.
7. For end-to-end verification, follow the same real-install methodology used in this effort: real `apt install`, run `horizon-grid configure` to drive the setup wizard through a real `docker compose up -d --build`, and exercise the CLI commands (`status`, `backup`, `diagnostics`, `stop` / `start`, `remove` / `purge`) against the resulting install. If testing `apt purge` / `apt remove` cleanup logic specifically with the real default project name, do so inside an isolated environment (e.g., a dedicated Docker-in-Docker daemon) to avoid colliding with any other running instance on a shared host, exactly as done in this session's testing.
8. Version numbers (package version, `MyAppVersion`, backend, frontend) must be kept in lockstep across platforms — the current release is `0.2.2` on both Windows and Linux, with no version drift.